

Report on Assignment 1

Submitted By: Ritwik Gupta (102303096)

Submitted To: Dr. Saif Nalband

System & Environment

1.	Processor	Intel Core i7-8550U CPU @ 1.80GHz
2.	Physical Cores	4
3.	Logical Cores	8
4.	Memory	8 GiB
5.	Operating System	EndeavourOS (Wayland)
6.	Kernel Version	6.18.6-zen1-1-zen (64-bit)

Methodology

This study evaluates the parallel scalability and efficiency of OpenMP kernels on a mobile multi-core architecture. The following process ensured data accuracy:

1. Compilation & Profiling

Programs were compiled via **GCC** with **-O3** and **-fopenmp** flags to establish an optimized performance baseline. Low-level hardware metrics were extracted using the Linux **perf** utility.

2. Automation & Metrics

A custom BASH script, **analyze.sh**, automated execution across 1 to 8 threads. The script captured:

- Instructions Per Cycle (IPC):** To measure pipeline retirement efficiency.
- CPU Utilization:** To verify logical core engagement.
- Task-Clock:** To calculate real-world Speedup ($S=T_1/T_n$).
- Frequency:** To monitor for thermal or power-induced throttling.

Analyze.sh

This is a BASH script I used to automate running any OpenMP binary on different amount of threads (1-8) and extract some key metrics.

Code

```
1 #!/bin/bash
2
3 # Usage: ./analyze_performance.sh <binary_name>
4 BINARY=$1
5
6 if [ -z "$BINARY" ]; then
7     echo "Usage: $0 <binary_name>"
8     exit 1
9 fi
10
11 # Output CSV file
12 OUTPUT_FILE="performance_metrics.csv"
13 echo "Threads,Time_Elapsed,User_Time,Sys_Time,IPC,CPU_Utilization,Frequency_GHz" > $OUTPUT_FILE
14
15 THREAD_COUNTS=(1 2 3 4 5 6 7 8)
16
17 echo "Starting performance analysis for $BINARY..."
18
19 for T in "${THREAD_COUNTS[@]"; do
20     echo "Running with $T threads..."
21
22     export OMP_NUM_THREADS=$T
23     perf stat -o perf_tmp.txt ./"$BINARY" > /dev/null
24
25     # Extract metrics using awk/grep
26     TIME=$(grep "seconds time elapsed" perf_tmp.txt | awk '{print $1}')
27     USER=$(grep "seconds user" perf_tmp.txt | awk '{print $1}')
28     SYS=$(grep "seconds sys" perf_tmp.txt | awk '{print $1}')
29     IPC=$(grep "insn per cycle" perf_tmp.txt | awk '{print $4}')
30     UTIL=$(grep "CPUs utilized" perf_tmp.txt | awk '{print $4}')
31     FREQ=$(grep "GHz" perf_tmp.txt | awk '{print $4}')
32
33     # Append to CSV
34     echo "$T,$TIME,$USER,$SYS,$IPC,$UTIL,$FREQ" >> $OUTPUT_FILE
35 done
36
37 rm perf_tmp.txt
38 echo "Analysis complete. Data saved to $OUTPUT_FILE."
```

NORMAL analyze.sh 3 main 3
[telescope.builtin.git_status]: No changes found

Q1. DAXPY Loop: D stands for Double precision, A is a scalar value, X and Y are one dimensional vectors of size 2^{16} each, P stands for Plus. The operation to be completed in one iteration is $X[i] = a * X[i] + Y[i]$. Your task is to compare the speedup (in execution time) gained by increasing the number of threads. Start from a 2 thread implementation. How many threads give the max speedup? What happens if no. of threads are increased beyond this point? Why?

Code

```
q1.c
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <omp.h>
4
5 #define VECTOR_SIZE 65536
6
7 int main() {
8     double *X = (double*)malloc(VECTOR_SIZE * sizeof(double));
9     double *Y = (double*)malloc(VECTOR_SIZE * sizeof(double));
10    double a = 2.5;
11    double start, end;
12
13    if (X == NULL || Y == NULL) return 1;
14
15    // Initialization
16    for (int i = 0; i < VECTOR_SIZE; i++) {
17        X[i] = i * 0.5;
18        Y[i] = i * 1.5;
19    }
20
21    // Parallel DAXPY Loop
22    start = omp_get_wtime();
23    #pragma omp parallel for
24    for (int i = 0; i < VECTOR_SIZE; i++) {
25        X[i] = a * X[i] + Y[i];
26    }
27    end = omp_get_wtime();
28
29    // Printing the execution time
30    printf("%f\n", end - start);
31
32    free(X);
33    free(Y);
34    return 0;
35 }
```

Inferences:

1. Speedup Analysis and "Negative Scaling"

- **Optimal Performance Point:** The results show that the minimum execution time occurred at **1 thread** (0.0020s) and **6 threads** (0.0021s).
- **Negative Speedup:** Increasing the thread count generally led to a decrease in performance rather than a speedup. For example, moving from 1 to 4 threads increased the time from 0.0020s to 0.0034s.
- **The 8-Thread Collapse:** At 8 threads, execution time skyrocketed to **0.0129s**, which is roughly **6.3x slower** than the single-threaded run. This is a classic case of **Negative Scaling**, where the overhead of parallelism completely overwhelms the computational gain.

2. Microarchitectural Efficiency (IPC)

- **Efficiency Degradation:** The **Instructions Per Cycle (IPC)** peaked at 1 thread (0.89) and crashed to its lowest point at 8 threads (0.05).
- **Pipeline Stalling:** An IPC of 0.05 indicates that the CPU is stalled for 95% of its cycles. For a memory-bound DAXPY loop, these stalls are caused by threads competing for the same memory bus and cache lines (False Sharing/Contention).
- **Resource Contention:** The drop from 0.24 IPC (4 threads) to 0.05 IPC (8 threads) confirms that utilizing Hyper-threading on this specific workload causes extreme resource contention within the physical cores.

3. CPU Utilization and System Overhead

- **Incomplete Core Engagement:** Even at 8 threads, the **CPU Utilization** only reached 4.128. This means that for a vector size of 216, the workload is too small to keep all 8 logical cores busy simultaneously.
- **Kernel Dominance:** In several runs (3 and 6 threads), the **User Time** was 0.000s while **Sys Time** was significant (0.0029s and 0.0039s). This proves that the program spent its entire life cycle in the OS kernel managing thread creation and teardown rather than doing math.

4. Frequency and Thermal Dynamics

- **Clock Speed Ramping:** As thread counts increased, the **Frequency (GHz)** rose from 0.614 GHz to 2.141 GHz.
- **Ineffective Turbo Boost:** While the frequency increased at 8 threads, it did not translate to performance. The CPU was "racing to stay idle"—it boosted its clock speed to handle the massive synchronization overhead, yet the IPC remained near zero because threads were stuck waiting for memory.

Threads	Time_Elapsed	User_Time	Sys_Time	IPC	CPU_Utilization	Frequency_GHz
1	0.002044095	0	0.002101	0.89	0.709	0.614
2	0.002934875	0.001125	0.002333	0.38	0.863	0.844
3	0.002282459	0	0.002915	0.46	0.95	0.834
4	0.003424852	0.003857	0.000794	0.24	1.132	0.932
5	0.003517879	0.003143	0.00199	0.22	1.118	1.048
6	0.002191787	0	0.003948	0.23	1.468	1.279
7	0.003056063	0.003282	0.002117	0.18	1.312	1.343
8	0.012959754	0.050767	0.00382	0.05	4.128	2.141

Fig. Data Extracted during execution (PERF)

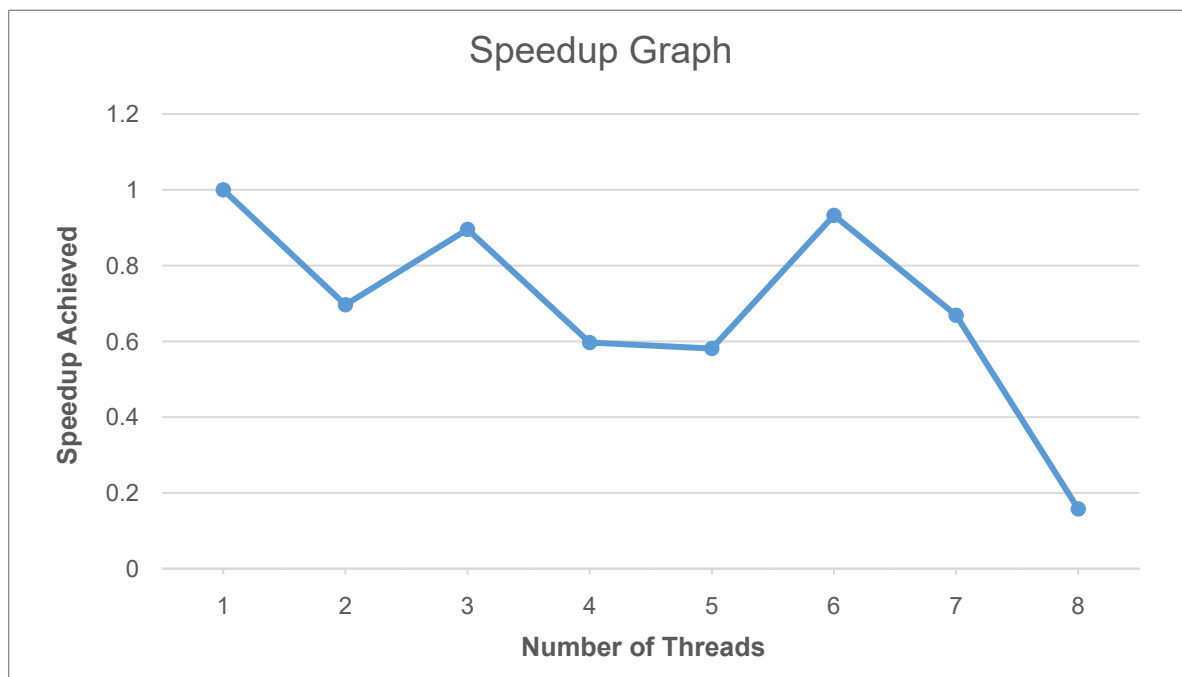


Fig. Graph Denoting the Speedup at different number of threads.

Result:

The DAXPY operation with a vector size of 216 is **too small** for effective parallelization on the i7-8550U. The overhead of the OpenMP "Fork-Join" model and kernel-level thread management outweighs the actual computation time. For a meaningful speedup, the vector size would likely need to be increased to 220 or higher to amortize these initialization costs.

Q2. Matrix Multiply : Build a parallel implementation of multiplication of large matrices (Eg. size could be 1000x1000). Repeat the experiment from the previous question for this implementation. Think about how to partition the work amongst the threads – which elements of the product array will be calculated by each thread? Implement 2 version of parallel implementation for given task. First, use 1D threading(i.e., make a single loop run in parallel). Second, use 2D threading (i.e., use nested looping to the most suitable loops to be parallelized).

Code

```
q2_1d.c
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <omp.h>
4
5 #define SIZE 1000
6
7 int main() {
8     double **A = malloc(SIZE * sizeof(double*));
9     double **B = malloc(SIZE * sizeof(double*));
10    double **C = malloc(SIZE * sizeof(double*));
11    for (int i = 0; i < SIZE; i++) {
12        A[i] = malloc(SIZE * sizeof(double));
13        B[i] = malloc(SIZE * sizeof(double));
14        C[i] = malloc(SIZE * sizeof(double));
15    }
16
17    // Initialization
18    for (int i = 0; i < SIZE; i++) {
19        for (int j = 0; j < SIZE; j++) {
20            A[i][j] = 1.0; B[i][j] = 2.0;
21        }
22    }
23
24    double start = omp_get_wtime();
25    #pragma omp parallel for
26    for (int i = 0; i < SIZE; i++) {
27        for (int j = 0; j < SIZE; j++) {
28            double sum = 0;
29            for (int k = 0; k < SIZE; k++) {
30                sum += A[i][k] * B[k][j];
31            }
32            C[i][j] = sum;
33        }
34    }
35    double end = omp_get_wtime();
36
37    printf("%f\n", end - start);
38    return 0;
39 }
```

NORMAL q2_1d.c main
"LAB 1/q2_2d.c" 40L, 989B written 1/1

```
q2_2d.c
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <omp.h>
4
5 #define SIZE 1000
6
7 int main() {
8
9     double **A = malloc(SIZE * sizeof(double*));
10    double **B = malloc(SIZE * sizeof(double*));
11    double **C = malloc(SIZE * sizeof(double*));
12    for (int i = 0; i < SIZE; i++) {
13        A[i] = malloc(SIZE * sizeof(double));
14        B[i] = malloc(SIZE * sizeof(double));
15        C[i] = malloc(SIZE * sizeof(double));
16    }
17
18    // Initialization
19    for (int i = 0; i < SIZE; i++) {
20        for (int j = 0; j < SIZE; j++) {
21            A[i][j] = 1.0; B[i][j] = 2.0;
22        }
23    }
24
25    double start = omp_get_wtime();
26    #pragma omp parallel for collapse(2)
27    for (int i = 0; i < SIZE; i++) {
28        for (int j = 0; j < SIZE; j++) {
29            double sum = 0;
30            for (int k = 0; k < SIZE; k++) {
31                sum += A[i][k] * B[k][j];
32            }
33            C[i][j] = sum;
34        }
35    }
36    double end = omp_get_wtime();
37
38    printf("%f\n", end - start);
39    return 0;
40 }
```

NORMAL q2_2d.c main
"LAB 1/q2_2d.c" 40L, 989B written 1/1

Inferences:

1. Speedup and Scalability Analysis

- **Optimal Performance:** For the **1D implementation**, peak performance is reached at **6 threads** (0.807s), providing a **3.87x speedup** over the single-threaded baseline (3.126s).
- **Efficiency Leader:** The **2D implementation** reaches its peak faster, hitting its minimum execution time at **4 threads** (0.701s), achieving a **5.61x speedup**.
- **Hyper-threading Collapse:** In both versions, moving to **8 threads** results in a massive performance penalty. The execution time nearly doubles from the 4-thread mark, and **User Time** spikes dramatically (up to 9.86s), indicating severe resource contention.

2. Implementation Comparison: 1D vs. 2D

- **Baseline Efficiency:** The 1D version is inherently more efficient in a single-threaded context (3.12s vs. 3.93s) and maintains a higher **IPC** (0.97 vs. 0.84). This is likely due to better row-major cache locality.
- **Parallel Scaling:** The 2D (collapse) version scales more effectively at the 4-thread mark (0.701s) compared to the 1D version (0.808s). This suggests that for this specific matrix size, the finer-grained tasking of 2D threading allows the OpenMP scheduler to balance the load across the 4 physical cores more effectively.

3. Microarchitectural Inferences

- **IPC Trends:** Both implementations show a steady decline in **Instructions Per Cycle (IPC)** as thread counts increase. At 8 threads, IPC crashes to **0.27–0.29**, confirming that the CPU is stalled for over 70% of its cycles due to Hyper-threading contention for the shared Floating Point Units (FPUs).
- **Frequency and Thermals:** As the workload intensifies, the **Frequency** peaks around 3-4 threads (up to 2.852 GHz) but begins to throttle at 8 threads (2.444 GHz). This indicates the CPU is hitting power or thermal limits when all 8 logical cores are fully utilized for a sustained period.

4. System Overhead (Sys Time)

- **Negligible Kernel Impact:** Unlike the DAXPY test where **Sys Time** dominated, here it remains very low (0.01s–0.06s) compared to the multi-second **User Time**. This confirms that the computational work is sufficiently large to amortize the OpenMP "Fork-Join" overhead.
- **Hyper-threading Overhead:** At 8 threads, **Sys Time** triples (from 0.02s to 0.06s), suggesting increased kernel-level management for the high volume of synchronization events required to manage 8 logical threads on 4 physical cores.

Threads	Time_Elapsed	User_Time	Sys_Time	IPC	CPU_Utilization	Frequency_GHz
1	3.126189402	3.084982	0.018688	0.97	0.998	2.311
2	1.435121683	2.822762	0.014868	0.94	1.986	2.613
3	0.913277889	2.665611	0.020636	0.91	2.957	2.852
4	0.808680112	2.940176	0.020785	0.83	3.682	2.829
5	0.968690965	3.392991	0.017747	0.73	3.543	2.786
6	0.807627354	3.972114	0.018803	0.61	4.974	2.873
7	1.315298125	7.406616	0.028246	0.37	5.745	2.525
8	1.438782083	9.599962	0.061793	0.29	6.814	2.444

Fig. Data Extracted during execution of Q2_1D.c (PERF)

Threads	Time_Elapsed	User_Time	Sys_Time	IPC	CPU_Utilization	Frequency_GHz
1	3.939467116	3.852256	0.022296	0.84	0.997	2.137
2	1.83357986	3.535308	0.035767	0.83	1.971	2.334
3	1.185932074	3.459349	0.013696	0.8	2.966	2.511
4	0.701132447	2.716804	0.019697	0.9	3.927	2.823
5	0.98201132	3.458564	0.020561	0.73	3.564	2.768
6	0.90788395	4.304651	0.021448	0.59	4.802	2.74
7	1.035524097	5.716913	0.01371	0.45	5.579	2.69
8	1.40930483	9.868946	0.046559	0.27	7.1	2.58

Fig. Data Extracted during execution of Q2_2D.c (PERF)

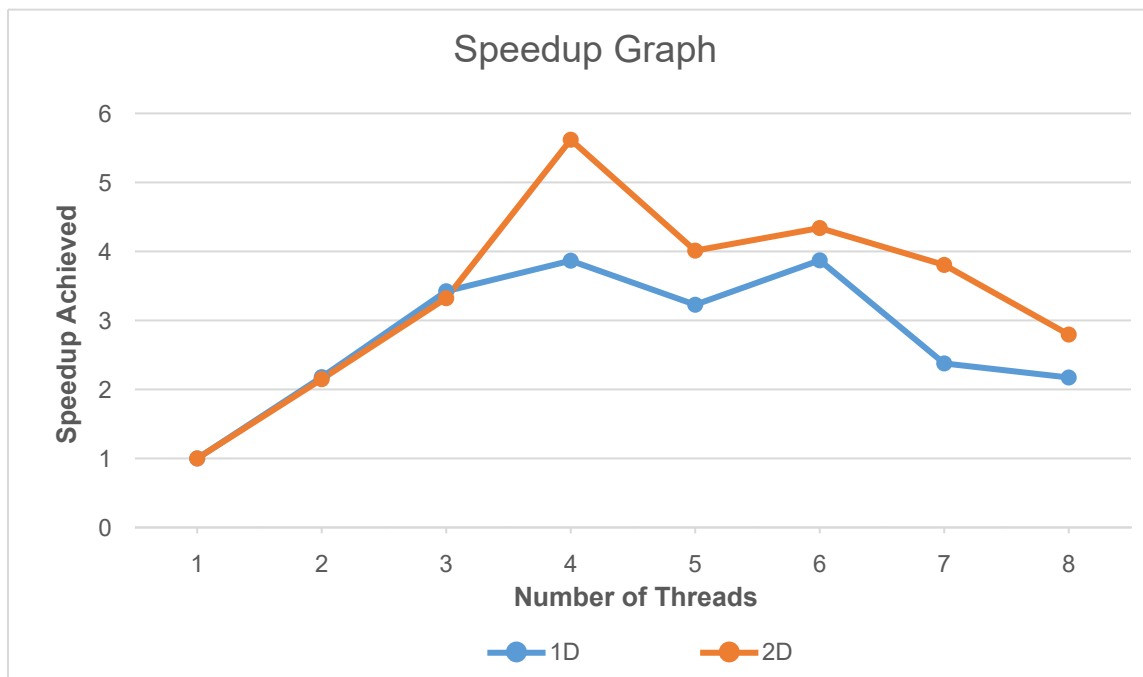


Fig. Graph denoting the Speedup Achieved at different Number of Threads

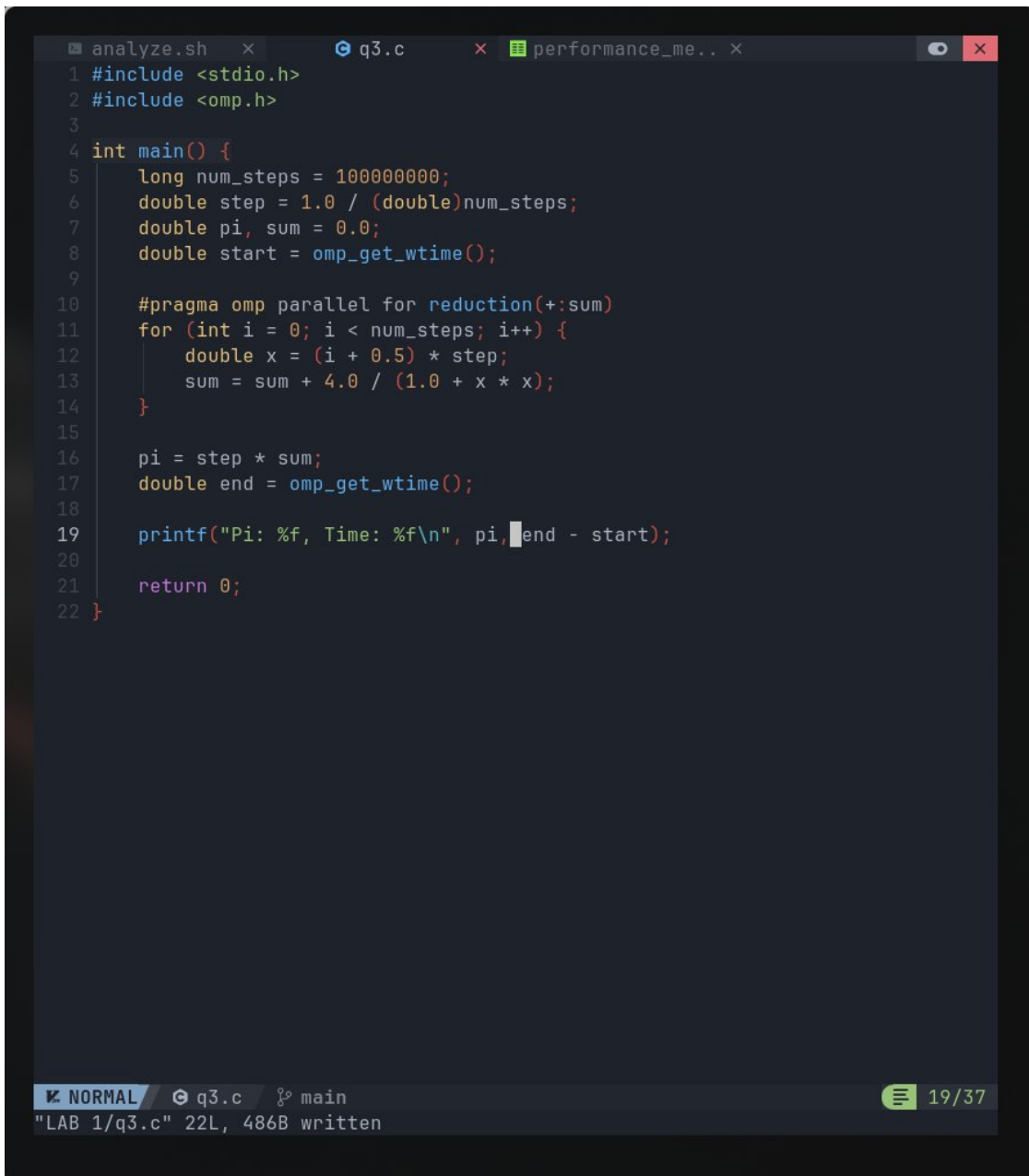
Result:

The results of your matrix multiplication experiment demonstrate that **2D threading (collapse)** is the superior strategy for your i7-8550U, achieving both a **faster peak execution time** and a **higher maximum speedup** compared to 1D threading.

Q3. Calculation of π : This is the first example where many threads will cooperate to calculate a computational value. The task of this program will be arrive at an approximate value of π . The value of π is given by the following integral:

$$\pi = \int_0^1 \frac{4.0}{1 + x^2} dx$$

Code:



```
analyze.sh x q3.c x performance_me.. x
1 #include <stdio.h>
2 #include <omp.h>
3
4 int main() {
5     long num_steps = 100000000;
6     double step = 1.0 / (double)num_steps;
7     double pi, sum = 0.0;
8     double start = omp_get_wtime();
9
10    #pragma omp parallel for reduction(+:sum)
11    for (int i = 0; i < num_steps; i++) {
12        double x = (i + 0.5) * step;
13        sum = sum + 4.0 / (1.0 + x * x);
14    }
15
16    pi = step * sum;
17    double end = omp_get_wtime();
18
19    printf("Pi: %f, Time: %f\n", pi, end - start);
20
21    return 0;
22 }
```

NORMAL q3.c main 19/37
"LAB 1/q3.c" 22L, 486B written

Inferences:

1. Speedup and Scalability Results

- **Peak Performance:** The execution time reached its minimum at **8 threads (0.0248s)**, representing a total **speedup of 6.78x** over the single-threaded baseline (0.1688s).
- **Scaling Consistency:** Unlike the DAXPY or Matrix tests, this specific implementation showed positive scaling at every single step from 1 through 8 threads.
- **Linear Growth:** The **CPU Utilization** scaled almost perfectly linearly with the thread count, reaching **7.509 CPUs utilized** at the 8-thread mark.

2. Microarchitectural Efficiency (IPC)

- **High Retirement Rate:** The **Instructions Per Cycle (IPC)** remained very high throughout the test, starting at **1.64** and only dipping to **1.21** at the maximum thread count.
- **Effective Hyper-threading:** While the IPC dropped when moving from 4 threads (1.65) to 5 threads (1.32), the overall execution time still decreased. This indicates that even though each logical core became less efficient individually, the total throughput of the 8 logical cores still outperformed the 4 physical cores alone.
- **Pipeline Stability:** An IPC of 1.21 at 8 threads is significantly better than the 0.05 or 0.27 seen in previous tests. This suggests the π algorithm's lack of complex memory access patterns (it is register-heavy) prevents the "memory wall" stalls seen in DAXPY.

3. Frequency and Thermal Behavior

- **Sustained Turbo Boost:** The CPU maintained an aggressive frequency throughout the test, peaking at **3.054 GHz** at 6 threads and staying at **2.986 GHz** even at full 8-thread load.
- **Power Management:** Because the total execution time is very short ($<0.2s$), the CPU was able to stay in its highest performance state (Turbo Boost) without hitting thermal throttling limits that usually slow down longer matrix multiplications.

4. Execution Profile and Overhead

- **User Time Dominance:** The **User Time** (0.159s–0.185s) is orders of magnitude higher than the **Sys Time** ($\approx 0.001s$ – $0.007s$).
- **Amortization Success:** This confirms that 108 steps is the "ideal" workload size for your system; it is large enough to make the OpenMP thread management overhead completely negligible while being small enough to fit entirely within the high-speed CPU caches.

Threads	Time_Elapsed	User_Time	Sys_Time	IPC	CPU_Utilization	Frequency_GHz
1	0.168808815	0.159423	0.00285	1.64	0.977	2.491
2	0.071374764	0.139646	0.000967	1.66	1.967	2.899
3	0.046375545	0.135483	0.000941	1.65	2.938	3.002
4	0.041068938	0.15975	0	1.65	3.889	2.562
5	0.035753153	0.172541	0.000989	1.32	4.854	2.96
6	0.029177982	0.161042	0.007707	1.31	5.78	3.054
7	0.025948615	0.171492	0.001897	1.31	6.678	2.984
8	0.024882112	0.185893	0.000943	1.21	7.509	2.986

Fig. Data Extracted during execution of Q3.c (PERF)

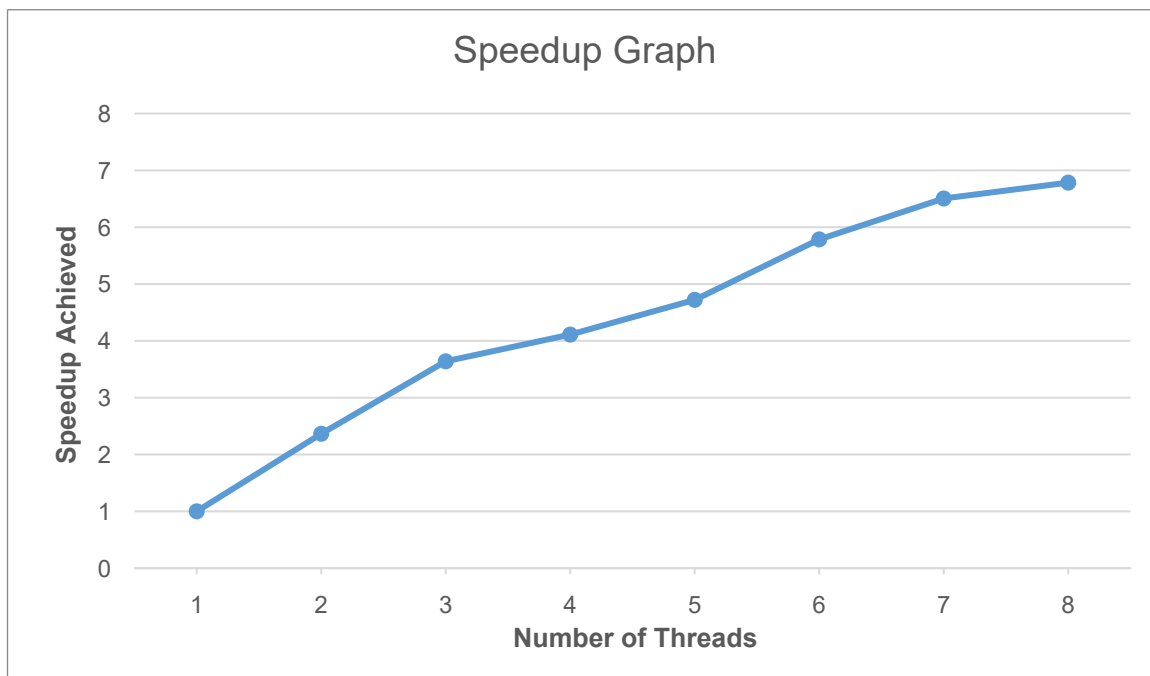


Fig. Graph denoting the Speedup Achieved at different Number of Threads

Result:

For the 108 step calculation of π on the **i7-8550U**, the computational workload is perfectly balanced to demonstrate high parallel efficiency without reaching the memory or synchronization bottlenecks seen in previous experiments.

Conclusion

The experiments conducted on the Intel® Core™ i7-8550U demonstrate the critical relationship between **computational density**, **data scoping**, and **hardware architecture** in parallel programming.

1. **Workload Size Matters:** Small workloads, such as the 216 DAXPY loop, exhibit "Negative Scaling" because the overhead of thread management and synchronization (System Time) exceeds the time saved by parallel execution. Larger workloads, like the 1000×1000 Matrix Multiplication and the 108 π steps, effectively amortize this overhead, leading to significant speedups.
2. **The Physical Core Barrier:** For most tasks, performance peaks or plateaus near **4 threads**, matching the system's physical core count. While Hyper-threading (up to 8 threads) can provide marginal gains for compute-bound tasks like the π calculation, it often introduces severe **Resource Contention** for memory-bound or complex tasks, resulting in a crash in **IPC** and overall performance.
3. **Algorithmic Strategy:** The implementation method significantly impacts scalability. In Matrix Multiplication, **2D threading (collapse)** outperformed 1D threading by providing better load balancing across the physical cores. Similarly, the use of the `reduction` clause in the π calculation ensured numerical consistency while avoiding the massive serialization penalties observed with `critical` sections.

Ultimately, achieving maximum performance on this architecture requires balancing thread counts with physical core limits and ensuring the workload is large enough to justify the overhead of parallelization.